```
PROGRAM PR2_Leak_Detection
VAR
    // --- Hardware Inputs (Mapped to Beckhoff ELM3148) ---
    iRawAnalogInput AT %I* : INT;   // Raw 4-20mA input from DLS panel
    bStartTestCycle        : BOOL;  // Triggered by FAT rig operator to begin leak test

    // --- System Configuration Constants (1kg Top-Down Architecture) ---
    c_iRawMin          : INT := 0;      // Raw integer value at 4mA (Beckhoff specific, often 0)
    c_iRawMax          : INT := 32767;  // Raw integer value at 20mA (Beckhoff specific 16-bit)
    c_rSensorCapacity  : REAL := 1000.0;// 1kg load cell (in grams)
    c_tSettleTime      : TIME := T#3S;  // 3-second mechanical deadband for suspension swing

    // --- Internal Logic Variables ---
    rAbsoluteMass      : REAL;  // Total mass currently hanging on the sensor
    rTareMass          : REAL;  // The captured weight of the empty bottle (approx 54g)
    bTareComplete      : BOOL;  // Flag indicating system is zeroed and ready
    rActiveLeakMass    : REAL;  // The calculated leak volume

    // --- Function Blocks ---
    fbSettleTimer      : TON;   // Timer for the deadband
    fbTareTrigger      : R_TRIG;// Rising edge trigger to capture tare once
END_VAR

// ==========================================================================================
// 1. Analog Input Scaling (4-20mA to Grams)
// ==========================================================================================
// Convert the raw Beckhoff integer into a percentage, then multiply by the 1000g capacity.
IF c_iRawMax > c_iRawMin THEN
    rAbsoluteMass := (INT_TO_REAL(iRawAnalogInput - c_iRawMin) / INT_TO_REAL(c_iRawMax - c_iRawMin)) *
c_rSensorCapacity;
ELSE
    rAbsoluteMass := 0.0; // Prevent divide by zero error
END_IF

// ==========================================================================================
// 2. Mechanical Deadband & Tare Logic
// ==========================================================================================
// Start the settling timer only when the operator begins the test cycle.
fbSettleTimer(IN := bStartTestCycle, PT := c_tSettleTime);

// When the timer finishes, trigger the tare capture ONCE.
fbTareTrigger(CLK := fbSettleTimer.Q);

// Execute the Tare: Store the absolute mass into the memory variable.
IF fbTareTrigger.Q THEN
    rTareMass := rAbsoluteMass;
    bTareComplete := TRUE;
END_IF

// Reset logic if test is stopped
IF NOT bStartTestCycle THEN
    bTareComplete := FALSE;
    rTareMass := 0.0;
```

```
    rActiveLeakMass := 0.0;
END_IF

// ================================================================================
// 3. Active Leak Measurement (1g = 1ml)
// ================================================================================
// Only calculate the leak if the system has successfully tared the 54g bottle.
IF bTareComplete THEN
    // Active leak is current mass minus the stored tare mass.
    rActiveLeakMass := rAbsoluteMass - rTareMass;

    // Optional: Implement a software filter to prevent negative readings from ambient vibration
    IF rActiveLeakMass < 0.0 THEN
        rActiveLeakMass := 0.0;
    END_IF
ELSE
    rActiveLeakMass := 0.0; // Output is zero while stabilizing
END_IF

// Note: rActiveLeakMass now directly represents milliliters of leaked fluid (PR2 data).
```